

数据并行(Data parallelism)指的是在数据集的不同部分上执行的计算工作可以相互独立地进行，因此可以并行执行。许多应用程序表现出丰富的数据并行性，使它们适用于可扩展的并行执行。因此，对于并行程序员来说，熟悉数据并行性的概念以及用于编写利用数据并行性的代码的并行编程语言结构是非常重要的。在本章中，我们将使用CUDA C语言结构来开发一个简单的数据并程序。

2.1 数据并行性

当现代软件应用运行缓慢时，问题通常出在数据上——数据量太大而难以处理。图像处理应用处理包含数百万到数万亿像素的图像或视频。科学应用使用数十亿的网格点来模拟流体动力学。分子动力学应用程序必须模拟数千亿个原子之间的相互作用。航空公司排班涉及数千个航班、机组人员和机场门。这些像素、粒子、网格点、相互作用、航班等等中的大多数通常可以被独立处理。例如，在图像处理中，将彩色像素转换为灰度只需要该像素的数据。模糊图像会将每个像素的颜色与附近像素的颜色平均，只需要该小邻域像素的数据。即使是看似全局的操作，例如找到图像中所有像素的平均亮度，也可以分解为许多较小的计算，这些计算可以相互独立地执行。这种对不同数据片段的独立评估是数据并行性的基础。编写数据并行代码涉及（重新）组织计算，使其围绕数据执行，以便我们可以并行执行生成的独立计算，以更快地完成整体工作——通常快得多。

让我们通过一个彩色转灰度的示例来说明数据并行性的概念。图2.1显示了一个彩色图像（左侧），由许多像素组成，每个像素包含一个从0（黑色）到1（完全强度）变化的红色、绿色和蓝色分数值（ r 、 g 、 b ）。



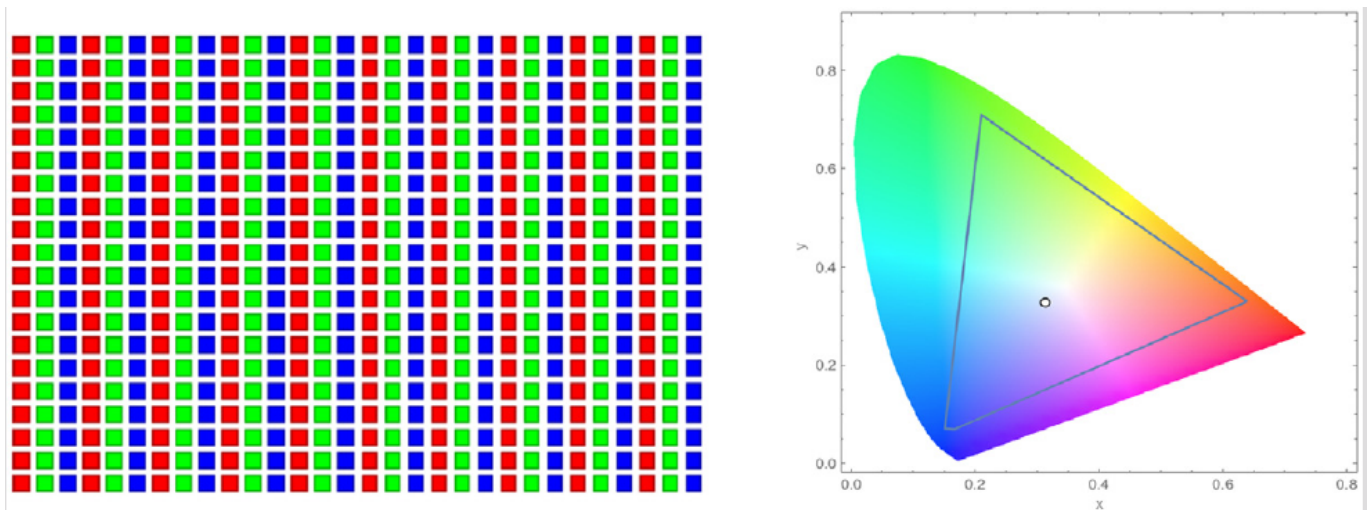
图2.1 把异构彩色图像转换为灰度图

为了将彩色图像（图2.1左侧）转换为灰度图像（右侧），我们通过应用以下加权和公式计算每个像素的亮度值 L ：

$$L = 0.21r + 0.72g + 0.07b$$

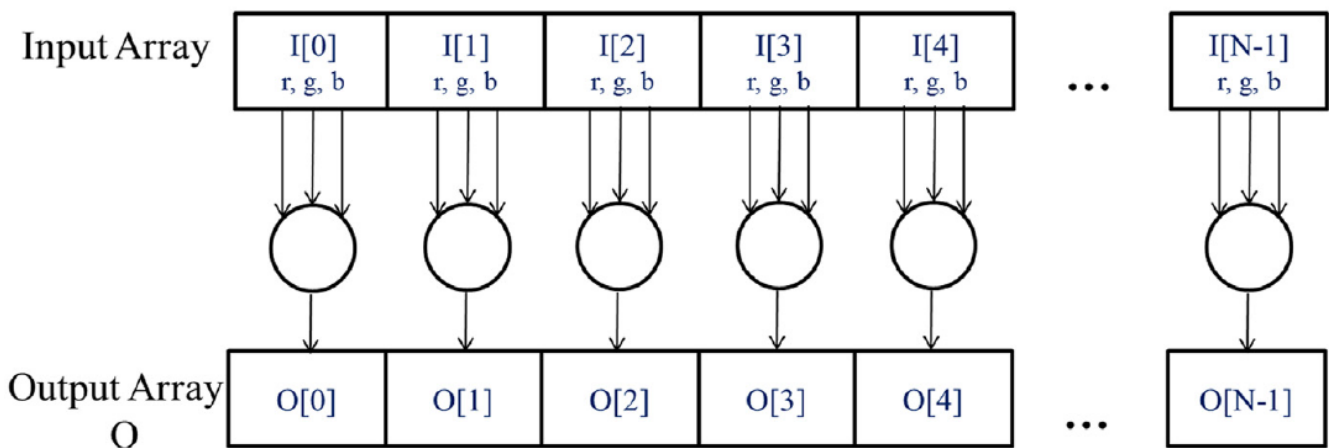
RGB色彩图像表示

在RGB表示中，图像中的每个像素都以 (r, g, b) 值的元组形式存储。图像的行的格式为 $(r, g, b) (r, g, b) \dots (r, g, b)$ ，如下概念图片所示。每个元组指定了红色（ R ）、绿色（ G ）和蓝色（ B ）的混合。也就是说，对于每个像素， r 、 g 和 b 的值表示在呈现像素时红色、绿色和蓝色光源的强度（0表示黑暗，1表示完全强度）。



这三种颜色的实际允许混合方式在行业指定的色彩空间中有所不同。在这里，AdobeRGB色彩空间中这三种颜色的有效组合显示为三角形的内部。每种混合的垂直坐标（ y 值）和水平坐标（ x 值）显示了像素强度的分数应该是G和R。剩余的部分（ $1-y-x$ ）的像素强度应分配给B。为了呈现图像，每个像素的 r 、 g 、 b 值被用于计算像素的总强度（亮度）以及混合系数（ x 、 y 、 $1-y-x$ ）。

如果我们将输入视为以RGB值组织的图像数组 I ，将输出视为相应的亮度值数组 O ，我们得到如图2.2所示的简单计算结构。例如， $O[0]$ 是通过根据上述公式计算 $I[0]$ 中的RGB值的加权和生成的； $O[1]$ 是通过计算 $I[1]$ 中的RGB值的加权和生成的； $O[2]$ 是通过计算 $I[2]$ 中的RGB值的加权和生成的，依此类推。所有这些逐像素计算都彼此独立，可以独立执行。显然，从彩色到灰度的转换表现出丰富的数据并行性。当然，在完整的应用程序中，数据并行性可能更加复杂，本书的大部分内容致力于教授发现和利用数据并行性所需的并行思维。



图像转灰度的数据并行性。像素可以独立计算。

任务并行性与数据并行性 在并行编程中，数据并行性并不是唯一使用的并行性类型。任务并行性在并行编程中也被广泛使用。任务并行性通常通过对应用程序的任务分解来暴露。例如，一个简单的应用程序可能需要进行矢量加法和矩阵-矢量乘法。其中每个都将是一个任务。如果这两个任务可以独立完成，那么就存在任务并行性。I/O和数据传输也是任务的常见来源。在大型应用程序中，通常存在更多独立的任务，因此也存在更多的任务并行性。例如，在分子动力学模拟器中，自然任务列表包括振动力、旋转力、用于非键作用力的邻居识别、非键作用力、速度和位置，以及基于速度和位置的其他物理性质。

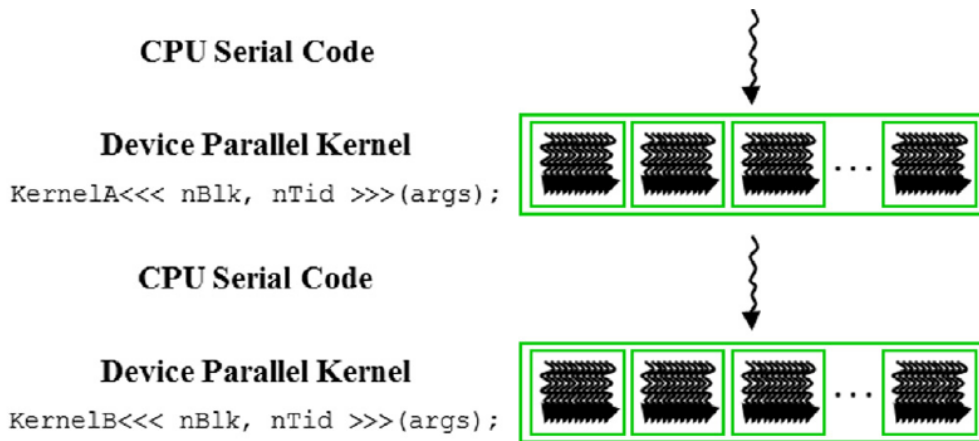
总体而言，数据并行性是并程序可伸缩性的主要来源。使用大型数据集，通常可以找到丰富的数据并行性，以便利用大规模并行处理器，并允许应用程序性能随着每一代硬件的到来而增长，因为后者具有更多的执行资源。然而，任务并行性在实现性能目标方面也可以发挥重要作用。当我们介绍流时，我们将涵盖任务并行性。

2.2 CUDA C程序结构

我们现在准备学习如何编写CUDA C程序，以利用数据并行性实现更快的执行。CUDA C通过最小的新语法和库函数将流行的ANSI C编程语言进行了扩展，使程序员能够针对包含CPU核心和大规模并行GPU的异构计算系统进行开发。顾名思义，CUDA C建立在NVIDIA的CUDA平台上。CUDA目前是最成熟的用于大规模并行计算的框架，被广泛应用于高性能计算行业，提供了在大多数常见操作系统上使用的编译器、调试器和性能分析工具等基本工具。

CUDA C程序的结构反映了计算机中主机（CPU）和一个或多个设备（GPU）的共存。每个CUDA C源文件可以包含主机代码和设备代码的混合。默认情况下，任何传统的C程序都是一个只包含主机代码的CUDA程序。可以将设备代码添加到任何源文件中。设备代码使用特殊的CUDA C关键字明确定义。设备代码包括函数或内核，其代码以数据并行方式执行。

CUDA程序的执行过程如图2.3所示。执行从主机代码（CPU串行代码）开始。当调用内核函数时，在设备上启动大量线程以执行内核。由内核调用启动的所有线程被集体称为一个网格。这些线程是CUDA平台中并行执行的主要工具。图2.3显示了两个线程网格的执行过程。我们将很快讨论这些网格是如何组织的。当一个网格的所有线程都完成执行时，该网格终止，并且执行继续在主机上，直到启动另一个网格。



CUDA程序的执行。

请注意，图2.3显示了一个简化的模型，其中CPU执行和GPU执行不重叠。许多异构计算应用程序管理重叠的CPU和GPU执行，以充分利用CPU和GPU的优势。

启动一个网格通常会生成许多线程，以利用数据并行性。在将颜色转为灰度的示例中，每个线程可以用于计算输出数组O的一个像素。在这种情况下，由网格启动生成的线程数等于图像中的像素数。对于大图像，将生成大量线程。CUDA程序员可以假设这些线程在生成和调度时需要很少的时钟周期，这归功于高效的硬件支持。这一假设与传统的CPU线程形成对比，后者通常需要数千个时钟周期来生成和调度。在接下来的章节中，我们将展示如何实现颜色转灰度和图像模糊内核。在本章的其余部分，我们将使用向量加法作为一个简单的运行示例。

线程 线程是现代计算机中处理器执行顺序程序的简化视图。一个线程包括程序的代码、正在执行的代码点以及其变量和数据结构的值。就用户而言，线程的执行是顺序的。用户可以使用源代码级调试器逐条执行语句，查看下一条将要执行的语句，并在执行过程中检查变量和数据结构的值。

线程在编程中已经使用了很多年。如果程序员希望在应用程序中启动并行执行，他/她可以使用线程库或特殊语言创建和管理多个线程。在CUDA中，每个线程的执行也是顺序的。CUDA程序通过调用内核函数启动并行执行，这会导致底层运行时机制启动一个处理不同数据部分的线程网格。

2.3 矢量加法内核

我们使用矢量加法来演示CUDA C程序的结构。矢量加法可以说是可能的数据并行计算中最简单的一个，是顺序编程中“Hello World”的并行等价物。在展示矢量加法的内核代码之前，先复习一下传统矢量加法（主机代码）函数的工作原理是有帮助的。图2.4展示了一个简单的传统C程序，包括一个主函数和一个矢量加法函数。在我们的所有示例中，每当需要区分主机和设备数据时，我们都会在主机使用的变量名称后缀“_h”，在设备使用的变量名称后缀“_d”，以提醒自己这些变量的预期用途。由于图2.4中只有主机代码，我们只看到后缀为“_h”的变量。

```
01 // Compute vector sum C_h = A_h + B_h
02 void vecAdd(float* A_h, float* B_h, float* C_h, int n) {
03     for (int i = 0; i < n; ++i) {
04         C_h[i] = A_h[i] + B_h[i];
05     }
06 }
07 int main() {
08     // Memory allocation for arrays A, B, and C
09     // I/O to read A and B, N elements each
10     ...
11     vecAdd(A, B, C, N);
12 }
```

简单的矢量加法示例

C语言中的指针

图2.4中的函数参数A、B和C都是指针。在C语言中，指针可用于访问变量和数据结构。例如，浮点变量V可以通过以下声明：

```
float V;
```

来声明。指针变量P可以通过以下声明：

```
float *P;
```

来声明。通过使用语句 $P = \&V$ 将V的地址赋给P，我们使P“指向”V。这样，P成为V的同义词。例如， $U = *P$ 将V的值赋给U。另一个例子， $*P = 3$ 将V的值更改为3。

在C程序中，可以通过指向数组的指针访问数组的第0个元素。例如，语句 $P = \&(A[0])$ 使P指向数组A的第0个元素。P[i] 成为A[i] 的同义词。事实上，数组名A本身就是指向其第0个元素的指针。

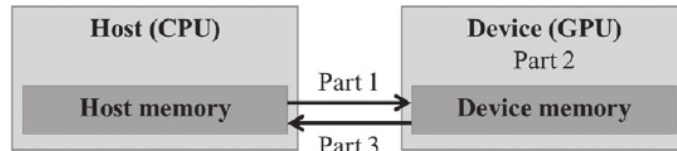
在图2.4中，将数组名A作为函数调用vecAdd的第一个参数传递给函数，使函数的第一个参数A_h指向A的第0个元素。因此，函数体中的 $A_h[i]$ 可用于访问主函数中数组A的 $A[i]$ 。

有关在C语言中指针的详细用法的易于理解的解释，请参见Patt & Patel（Patt & Patel, 2020）。

假设要相加的向量存储在在主程序中分配和初始化的数组A和B中。输出向量在数组C中，该数组也在主程序中分配。为简洁起见，我们没有显示在主函数中如何分配或初始化A、B和C的详细信息。这些数组的指针与包含向

量长度的变量N一起传递给vecAdd函数。请注意，vecAdd函数的参数后缀为“_h”，以强调它们由主机使用。在接下来的步骤中引入设备代码时，这种命名约定将会很有帮助。

图2.4中的vecAdd函数使用for循环遍历向量元素。在第i次迭代中，输出元素C_h[i]接收A_h[i]和B_h[i]的和。向量长度参数n用于控制循环，使得迭代次数与向量的长度相匹配。该函数通过指针A_h、B_h和C_h分别读取A和B的元素，并写入C的元素。当vecAdd函数返回时，主函数中的后续语句可以访问C的新内容。



```

01 void vecAdd(float* A, float* B, float* C, int n) {
02     int size = n* sizeof(float);
03     float *d_A *d_B, *d_C;
04
05     // Part 1: Allocate device memory for A, B, and C
06     // Copy A and B to device memory
07     ...
08
09     // Part 2: Call kernel - to launch a grid of threads
10     // to perform the actual vector addition
11     ...
12
13     // Part 3: Copy C from the device memory
14     // Free device vectors
15     ...
16 }
  
```

修改后的vecAdd函数的概要，将计算移到设备上。

在并行执行矢量加法的直接方法是修改vecAdd函数并将其计算移到设备上。修改后的vecAdd函数的结构如图2.5所示。函数的第一部分在设备（GPU）内存中分配空间以容纳A、B和C向量的副本，并将A和B向量从主机内存复制到设备内存。第二部分调用实际的矢量加法核函数，在设备上启动一个线程网格。第三部分将和向量C从设备内存复制到主机内存，并释放设备内存中的三个数组。请注意，修改后的vecAdd函数本质上是一个外包代理，将输入数据发送到设备，激活设备上的计算，并从设备收集结果。该代理以一种使主程序甚至无需知道矢量加法实际上是在设备上完成的方式执行此操作。实际上，由于数据的来回复制，这种“透明”外包模型通常效率较低。通常，人们会在设备上保留大型和重要的数据结构，并仅从主机代码中调用设备函数。然而，目前我们将使用简化的透明模型来介绍基本的CUDA C程序结构。修改后函数的详细信息，以及如何组合核函数，将是本章的主题。

2.4 设备全局内存和数据传输

在当前的CUDA系统中，设备通常是配有自己的动态随机访问内存（称为设备全局内存或全局内存）的硬件卡。例如，NVIDIA Volta V100配备了16GB或32GB的全局内存。将其称为“全局”内存是为了将其与程序员也可访问的其他类型的设备内存区分开。有关CUDA内存模型和不同类型的设备内存的详细信息，请参见第5章《内存架构和数据局部性》。

对于矢量加法核函数，在调用核函数之前，程序员需要在设备全局内存中分配空间并将数据从主机内存传输到设备全局内存中的已分配空间。这对应于图2.5的第一部分。同样，在设备执行后，程序员需要将结果数据从设备全局内存传输回主机内存，并释放在设备全局内存中分配的不再需要的空间。这对应于图2.5的第三部分。CUDA运行时系统（通常在主机上运行）提供了应用程序编程接口（API）函数，代表程序员执行这些活动。从这一点开始，我们将简单地说数据从主机传输到设备，以简称将数据从主机内存复制到设备全局内存中。相同的情况适用于相反的方向。

cudaMalloc()

- Allocates object in the device global memory
- Two parameters
 - **Address of a pointer** to the allocated object
 - **Size** of allocated object in terms of bytes

cudaFree()

- Frees object from device global memory
 - **Pointer** to freed object

图2.6 CUDA 管理设备全局内存的API函数。

在图2.5中，vecAdd函数的第一部分和第三部分需要使用CUDA API函数为A、B和C分配设备全局内存；将A和B从主机传输到设备；在矢量加法后将C从设备传输到主机；以及释放A、B和C的设备全局内存。首先，我们将解释内存分配和释放函数。

图2.6显示了两个用于分配和释放设备全局内存的API函数。cudaMalloc函数可以从主机代码中调用，为对象分配一块设备全局内存。读者应该注意cudaMalloc和标准C运行时库malloc函数之间的惊人相似之处。这是有意为之的；CUDA C是具有最小扩展的C。CUDA C使用标准C运行时库malloc函数来管理主机内存，同时将cudaMalloc添加为C运行时库的扩展。通过使接口尽可能接近原始C运行时库，CUDA C最小化了C程序员重新学习使用这些扩展的时间。

cudaMalloc函数的第一个参数是一个指针变量的地址，该变量将被设置为指向已分配对象的地址。指针变量的地址应强制转换为(void *)，因为该函数期望一个通用指针；内存分配函数是一个通用函数，不限于任何特定类型的对象。这个参数允许cudaMalloc函数将分配的内存的地址写入提供的指针变量，而不管其类型如何。调用核函数的主机代码将此指针值传递给需要访问已分配内存对象的核函数。cudaMalloc函数的第二个参数给出要分配的数据的大小，以字节为单位。该第二个参数的使用与C malloc函数的size参数一致。

注:

- CUDA C还具有更先进的库函数，用于在主机内存中分配空间。我们将在第20章“编程异构计算集群”中讨论它们。
- 事实上，cudaMalloc返回一个通用对象，这使得使用动态分配的多维数组更加复杂。我们将在第3.2节解决这个问题。
- 请注意，cudaMalloc与C的malloc函数具有不同的格式。C的malloc函数返回指向分配对象的指针。它只需要一个参数，指定分配对象的大小。而cudaMalloc函数写入作为第一个参数给出的指针变量的地址。因此，cudaMalloc函数需要两个参数。cudaMalloc的这种两参数格式使其能够使用返回值以与其他CUDA API函数相同的方式报告任何错误。

现在，我们使用以下简单的代码示例来说明cudaMalloc和cudaFree的用法：

```

#include <cuda_runtime.h>

int main() {
    // Declare device pointers
    float *A_d, *B_d, *C_d;

    // Allocate space in the device global memory for A
    cudaMalloc((void**)&A_d, size);

    // Your computation with A_d goes here

    // Free the allocated memory for A
    cudaFree(A_d);

    return 0;
}

```

这是对图2.5中示例的延续。为了清晰起见，我们使用“_d”后缀来表示指向设备全局内存中对象的指针变量。传递给cudaMalloc的第一个参数是指针A_d的地址（即&A_d）强制转换为void指针。当cudaMalloc返回时，A_d将指向为A向量分配的设备全局内存区域。传递给cudaMalloc的第二个参数是要分配的区域的大小。由于大小以字节数表示，程序员需要在确定大小时从数组中的元素数量转换为字节数。例如，在为包含n个单精度浮点元素的数组分配空间时，大小的值将是n乘以当今计算机上单精度浮点数的大小，即4字节。因此，大小的值将是n * 4。计算完成后，调用cudaFree，以A_d作为参数释放A向量的设备全局内存中的存储空间。请注意，cudaFree不需要更改A_d的值；它只需要使用A_d的值将分配的内存返回到可用池。因此，作为参数传递的是A_d的值而不是其地址。

A_d、B_d和C_d中的地址指向设备全局内存中的位置。这些地址不应在主机代码中取消引用。它们应在调用API函数和核函数中使用。在主机代码中取消引用设备全局内存指针可能会导致异常或其他类型的运行时错误。读者应该完成图2.5中vecAdd示例的Part 1，其中包含B_d和C_d指针变量的类似声明以及它们相应的cudaMalloc调用。此外，图2.5中的Part 3可以使用为B_d和C_d调用cudaFree来完成。

一旦主机代码为数据对象在设备全局内存中分配了空间，它可以请求将数据从主机传输到设备。这通过调用CUDA API函数之一来完成。图2.7展示了这样一个API函数，cudaMemcpy。cudaMemcpy函数有四个参数。第一个参数是指向要复制的数据对象目标位置的指针。第二个参数指向源位置。第三个参数指定要复制的字节数。第四个参数指示复制涉及的内存类型：从主机到主机，从主机到设备，从设备到主机以及从设备到设备。例如，内存复制函数可用于将数据从设备全局内存中的一个位置复制到设备全局内存中的另一个位置。

cudaMemcpy()

- memory data transfer
- Requires four parameters
 - Pointer to destination
 - Pointer to source
 - Number of bytes copied
 - Type/Direction of transfer

图2.7 CUDA API函数用于主机和设备之间的数据传输

vecAdd函数调用cudaMemcpy函数将A_h和B_h向量从主机内存复制到A_d和B_d在设备内存中，然后将它们相加，并在完成相加后将C_d向量从设备内存复制到C_h在主机内存中。假设A_h、B_h、A_d、B_d和size的值已经设

置好，下面是三个cudaMemcpy调用的示例。cudaMemcpyHostToDevice和cudaMemcpyDeviceToHost是CUDA编程环境中的已识别的预定义常量。请注意，通过正确排序源和目标指针并使用适当的常量进行传输类型，可以使用相同的函数在两个方向上传输数据。

```
cudaMemcpy(A_d, A_h, size, cudaMemcpyHostToDevice);
cudaMemcpy(B_d, B_h, size, cudaMemcpyHostToDevice);
// ...
cudaMemcpy(C_h, C_d, size, cudaMemcpyDeviceToHost);
```

总结一下，在图2.4中，主程序调用vecAdd，该函数也在主机上执行。vecAdd函数（见图2.5的概要）在设备全局内存中分配空间，请求数据传输，并调用执行实际矢量加法的核函数。我们将这种主机代码称为用于调用核函数的存根(stub)。在图2.8中，我们展示了vecAdd函数的更完整版本。

```
01 void vecAdd(float* A_h, float* B_h, float* C_h, int n) {
02     int size = n * sizeof(float);
03     float *A_d, *B_d, *C_d;
04
05     cudaMalloc((void **) &A_d, size);
06     cudaMalloc((void **) &B_d, size);
07     cudaMalloc((void **) &C_d, size);
08
09     cudaMemcpy(A_d, A_h, size, cudaMemcpyHostToDevice);
10     cudaMemcpy(B_d, B_h, size, cudaMemcpyHostToDevice);
11
12     // kernel invocation code - to be shown later
13     ...
14
15     cudaMemcpy(C_h, C_d, size, cudaMemcpyDeviceToHost);
16
17     cudaFree(A_d);
18     cudaFree(B_d);
19     cudaFree(C_d);
20 }
```

图2.8 更完整版本的vecAdd

与图2.5相比，图2.8中的vecAdd函数在第1部分和第3部分是完整的。第1部分为A_d、B_d和C_d分配设备全局内存，并将A_h传输到A_d和B_h传输到B_d。这通过调用cudaMalloc和cudaMemcpy函数完成。鼓励读者使用适当的参数值编写自己的函数调用，并将其代码与图2.8中显示的代码进行比较。第2部分调用核函数，将在下一小节中描述。第3部分将矢量和数据从设备复制到主机，以便在主函数中使用这些值。这通过调用cudaMemcpy函数完成。然后，它释放A_d、B_d和C_d在设备全局内存中的内存，这通过调用cudaFree函数实现（图2.9）。

CUDA中的错误检查和处理 通常，对于程序来说，检查并处理错误是很重要的。CUDA API函数在提供服务时返回指示错误是否发生的标志。大多数错误是由于调用中使用了不适当的参数值。为简洁起见，我们在示例中不会显示错误检查代码。例如，图2.9中显示了对cudaMalloc的调用：


```
cudaMalloc((void**) &A_d, size);
```

实际上，我们应该将调用包围在测试错误条件的代码中，并打印出错误消息，以使用户能够意识到发生了错误。这样的检查代码的简单版本如下：

```
cudaError_t err = cudaMalloc((void**) &A_d, size);
if (err != cudaSuccess) {
    printf("%s in %s at line %d\n",
        cudaGetErrorString(err),
        __FILE__, __LINE__);
    exit(EXIT_FAILURE);
}
```

这样，如果系统没有设备内存，用户将得到关于这种情况的通知。这可以节省许多调试时间。可以定义一个C宏，使源代码中的检查代码更简洁。

2.5 核函数和线程

我们现在准备更详细地讨论CUDA C核函数以及调用这些核函数的效果。在CUDA C中，核函数指定在并行阶段由所有线程执行的代码。由于所有这些线程执行相同的代码，CUDA C编程是众所周知的单程序多数据（SPMD）^{\$^1\$}（Atallah, 1998）并行编程风格的一个实例，这是一种流行的并行计算系统编程风格。

当程序的主机代码调用核函数时，CUDA运行时系统会启动一个线程网格，该网格组织成两级层次结构。每个网格都组织成一个线程块数组，我们将其简称为块。一个网格中的所有块都是相同大小的；每个块在当前系统上最多可以包含1024个线程^{\$^2\$}。图2.9显示了一个示例，其中每个块由256个线程组成。每个线程由从标有该线程在块中索引编号的框中出发的花括号箭头表示。

内建变量 许多编程语言都有内建变量。这些变量具有特殊的含义和目的。这些变量的值通常由运行时系统预初始化，并在程序中通常是只读的。程序员应该避免重新定义这些变量以供其他用途使用。

每个线程块中的总线程数是在调用核函数时由主机代码指定的。同一核函数可以在主机代码的不同部分以不同数量的线程调用。对于给定的线程网格，块中的线程数可以在名为`blockDim`的内建变量中找到。`blockDim`变量是一个结构，包含三个无符号整数字段（`x`、`y`和`z`），这些字段帮助程序员将线程组织成一维、二维或三维数组。对于一维组织，仅使用`x`字段。对于二维组织，使用`x`和`y`字段。对于三维结构，使用所有三个`x`、`y`和`z`字段。线程的组织方式通常反映数据的维度。这是有道理的，因为线程是为了并行处理数据而创建的，因此线程的组织方式自然应该反映数据的组织方式。在图2.9中，每个线程块被组织成一个一维数组的形式，因为数据是一维向量。`blockDim.x`变量的值表示每个块中的总线程数，在图2.9中为256。通常建议线程块的每个维度的线程数是32的倍数，出于硬件效率的原因。稍后我们将重新讨论这一点。

脚注：

- 1. 注意，SPMD并非与SIMD（单指令多数据）[Flynn 1972]相同。在SPMD系统中，并行处理单元在数据的多个部分上执行相同的程序。然而，这些处理单元不需要在同一时间执行相同的指令。在SIMD系统中，所有处理单元在任何时刻都执行相同的指令。
- 2. 在CUDA 3.0及更高版本中，每个线程块最多可以有1024个线程。一些较早的CUDA版本最多允许每个块有512个线程。

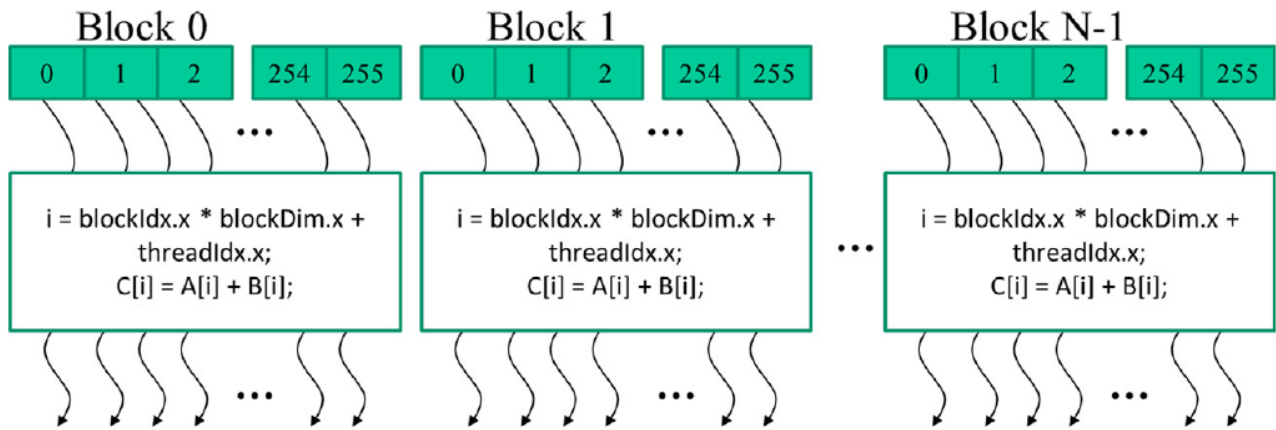


图2.9 所有网格中的线程执行相同的核函数代码。

CUDA核函数可以访问另外两个内建变量（`threadIdx`和`blockIdx`），这些变量允许线程彼此区分，并确定每个线程要处理的数据区域。`threadIdx`变量为每个线程提供块内的唯一坐标。在图2.9中，由于我们使用的是一维线程组织，仅使用`threadIdx.x`。图2.9中每个线程的`threadIdx.x`值显示在每个线程的小阴影框中。每个块中的第一个线程的`threadIdx.x`变量的值为0，第二个线程的值为1，第三个线程的值为2，依此类推。

分层组织 与CUDA线程一样，许多实际系统都是分层组织的。美国电话系统就是一个很好的例子。在顶层，电话系统由“区域”组成，每个区域对应一个地理区域。同一区域内的所有电话线都具有相同的3位“区号”。电话区域有时比城市大。例如，伊利诺伊州中部的许多县和城市都属于同一电话区域，并共享相同的区号217。在一个区域内，每条电话线都有一个七位数的本地电话号码，这使得每个区域最多可以拥有约一千万个号码。可以将每条电话线视为一个CUDA线程，其中区号是`blockIdx`的值，而七位本地号码是`threadIdx`的值。这种分层组织允许系统拥有大量的电话线，同时保留对同一区域进行呼叫的“局部性”。也就是说，在拨打同一区域内的电话时，呼叫者只需拨打本地号码。只要我们大多数时间都在本区域内拨打电话，我们很少需要拨打区号。如果我们偶尔需要拨打另一个区域的电话，我们拨打1和区号，然后是本地号码。（这就是为什么任何区域的本地号码都不应以1开头的原因。）CUDA线程的分层组织也提供了一种局部性形式。我们将很快研究这种局部性。

blockIdx变量为块中的所有线程提供一个共同的块坐标。在图2.9中，第一个块中的所有线程的`blockIdx.x`变量的值为0，第二个线程块中的值为1，依此类推。通过与电话系统的类比，可以将`threadIdx.x`视为本地电话号码，将`blockIdx.x`视为区号。两者一起为整个国家的每条电话线提供了唯一的电话号码。同样，每个线程可以将其`threadIdx`和`blockIdx`值结合起来，为自己在整个网格内创建唯一的全局索引。

在图2.9中，计算了一个唯一的全局索引 i ，即 $i = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$ 。回顾一下，我们的示例中`blockDim`的值为256。块0中线程的 i 值范围从0到255。块1中线程的 i 值范围从256到511。块2中线程的 i 值范围从512到767。也就是说，这三个块中线程的 i 值形成了从0到767的连续覆盖。由于每个线程使用 i 来访问A、B和C，这些线程涵盖了原始循环的前768次迭代。通过启动具有更多块的网格，可以处理更大的向量。通过启动具有 n 个或更多线程的网格，可以处理长度为 n 的向量。

```

01 // Compute vector sum C = A + B
02 // Each thread performs one pair-wise addition
03 __global__ ← kernel
04 void vecAddKernel(float* A, float* B, float* C, int n) {
05     int i = threadIdx.x + blockDim.x * blockIdx.x; index.
06     if (i < n) {
07         C[i] = A[i] + B[i];
08     }
09 }

```

*actual thread use
= #block * block-dim.
but not all thread
need to have task.
if not used, keep idle.*

*block within threads.
actual thread ID
thread ID + how many threads in
previous blocks.*

图2.10 向量加法核函数代码

图2.10显示了一个进行向量相加的核函数。请注意，在核函数中我们不使用“_h”和“_d”约定，因为这里没有潜在的混淆。在我们的示例中，核的语法是ANSI C，并带有一些显著的扩展。首先，在vecAddKernel函数的声明前面有一个CUDA-C特定的关键字“__global__”。此关键字表示该函数是一个核函数，可以调用它在设备上生成一个线程网格。

Qualifier Keyword	Callable From	Executed On	Executed By
<code>__host__</code> (default)	Host	Host	Caller host thread
<code>__global__</code>	Host (or Device)	Device	New grid of device threads
<code>__device__</code>	Device	Device	Caller device thread

图2.11 CUDA C函数声明的关键字

通常，CUDA C使用三个修饰关键字扩展了C语言，这些关键字可以在函数声明中使用。这些关键字的含义总结如图2.11所示。“__global__”关键字表示被声明的函数是一个CUDA C核函数。请注意，“__global__”一词两侧都有两个下划线字符。这样的核函数在设备上执行，并且可以从主机调用。在支持动态并行性的CUDA系统中，它也可以从设备调用，我们将在第21章“CUDA动态并行性”中看到。重要的特点是调用这样一个核函数会在设备上启动一个新的线程网格。

“__device__”关键字表示被声明的函数是CUDA设备函数。设备函数在CUDA设备上执行，只能从核函数或另一个设备函数调用。设备函数由调用它的设备线程执行，不会导致启动任何新的设备线程。\$^7\$

“__host__”关键字表示被声明的函数是CUDA主机函数。主机函数只是在主机上执行的传统C函数，只能从另一个主机函数调用。默认情况下，如果在其声明中没有任何CUDA关键字，则CUDA程序中的所有函数都是主机函数。这是有道理的，因为许多CUDA应用程序是从仅CPU执行环境迁移过来的。在迁移过程中，程序员会在主机函数中添加核函数和设备函数。原始函数仍然保留为主机函数。将所有函数默认为主机函数可以免去程序员修改所有原始函数声明的繁琐工作。

请注意，可以在函数声明中同时使用“__host__”和“__device__”。这种组合告诉编译系统为同一函数生成两个版本的目标代码。其中一个在主机上执行，只能从主机函数调用。另一个在设备上执行，只能从设备或核函数调用。这支持常见的用例，即相同的函数源代码可以重新编译以生成设备版本。许多用户库函数很可能属于这个类别。

C的第二个显著扩展，在图2.10中，是内建变量“threadIdx”、“blockIdx”和“blockDim”。回想一下，所有线程执行相同的核代码，它们需要一种方式来彼此区分并引导每个线程朝着数据的特定部分。这些内建变量是线程访问

提供给它们的标识坐标的硬件寄存器的手段。不同的线程将在它们的`threadIdx.x`、`blockIdx.x`和`blockDim.x`变量中看到不同的值。为了可读性，我们有时会在讨论中将一个线程称为`$thread_{blockIdx.x, threadIdx.x}$`。

图2.10中有一个自动（局部）变量`i`。在CUDA核函数中，自动变量对于每个线程都是私有的。也就是说，每个线程都会生成一个`i`的版本。如果使用10,000个线程启动网格，将会有10,000个版本的`i`，每个线程一个版本。由线程分配给其`i`变量的值对其他线程不可见。我们将在第5章“内存架构和数据局部性”中更详细地讨论这些自动变量。

通过将图2.4和图2.10进行快速比较，可以对CUDA核函数有一个重要的了解。图2.10中的核函数没有对应于图2.4中的循环。读者应该问循环去哪了。答案是循环现在被线程网格替代了。整个网格形成了循环的等效部分。网格中的每个线程对应于原始循环的一次迭代。这有时被称为循环并行性，其中原始顺序代码的迭代由线程并行执行。

请注意，图2.10中的`addVecKernel`函数有一个`if (i < n)`语句。这是因为并非所有的矢量长度都可以表示为块大小的倍数。例如，假设矢量长度是100。最小的有效线程块维度是32。假设我们选择32作为块大小。将需要启动四个线程块来处理所有100个矢量元素。然而，这四个线程块将有128个线程。我们需要禁用第3个线程块中的最后28个线程，以防它们执行原始程序不期望的工作。由于所有线程都将对它们的`i`值进行与`n`的比较，因此所有线程将测试它们的`i`值是否小于`n`，其中`n`是100。通过`if (i < n)`语句，前100个线程将执行加法，而最后的28个线程将不执行。这允许调用该核函数来处理任意长度的矢量。

脚注：

- 7 我们将在稍后解释在不同CUDA生成中使用间接函数调用和递归的规则。总的来说，为了实现最大的可移植性，应该避免在设备函数和核函数中使用递归和间接函数调用。

2.6 调用核函数

完成了核函数的实现后，剩下的步骤是从主机代码中调用该函数以启动网格。这在图2.12中进行了说明。当主机代码调用核函数时，它通过执行配置参数设置网格和线程块的维度。配置参数位于传统C函数参数之前的“<<<”和“>>>”之间。第一个配置参数给出了网格中的块数，第二个指定了每个块中的线程数。在这个例子中，每个块中有256个线程。为了确保我们有足够的线程在网格中覆盖所有的向量元素，我们需要将网格中的块数设置为所需线程数（在这种情况下为`n`）除以线程块大小（在这种情况下为256）的上取整（将商四舍五入为较高的整数值）。有许多执行上取整的方法。一种方法是对`n/256.0`应用C天花板函数。使用浮点值256.0确保我们生成一个浮点值，以便天花板函数可以正确地将其上取整。例如，如果我们想要1000个线程，我们将启动`ceil(1000/256.0) = 4`个线程块。因此，该语句将启动`4 * 256 = 1024`个线程。通过核函数中的`if (i, n)`语句（如图2.10所示），前1000个线程将对这1000个向量元素执行加法。剩下的24个将不执行。

```
01 int vectAdd(float* A, float* B, float* C, int n) {
02     // A_d, B_d, C_d allocations and copies omitted
03     ...
04     // Launch ceil(n/256) blocks of 256 threads each
05     vecAddKernel<<<ceil(n/256.0), 256>>>(A_d, B_d, C_d, n);
06 }
```

图2.12 调用核函数

function Name <<< # block, # thread-per-block >>> (input-param),


```

01 void vecAdd(float* A, float* B, float* C, int n) {
02     float *A_d, *B_d, *C_d;
03     int size = n * sizeof(float);
04
05     cudaMalloc((void **) &A_d, size);
06     cudaMalloc((void **) &B_d, size);
07     cudaMalloc((void **) &C_d, size);
08
09     cudaMemcpy(A_d, A, size, cudaMemcpyHostToDevice);  H->D
10     cudaMemcpy(B_d, B, size, cudaMemcpyHostToDevice);  H->D.
11
12     vecAddKernel<<<ceil(n/256.0), 256>>>(A_d, B_d, C_d, n); ← actual compute
13
14     cudaMemcpy(C, C_d, size, cudaMemcpyDeviceToHost);  D->H.
15
16     cudaFree(A_d);
17     cudaFree(B_d);
18     cudaFree(C_d);
19 }

```

图2.13 矢量加法完整的host代码

图2.13显示了vecAdd函数中的最终主机代码。此源代码完成了图2.5中的骨架。图2.12和图2.13共同说明了一个由主机代码和设备核函数组成的简单CUDA程序。该代码被硬编码为使用每个线程块256个线程。^{\$^8\$}然而，使用的线程块数量取决于向量的长度（ n ）。如果 n 为750，则将使用三个线程块。如果 n 为4000，则将使用16个线程块。如果 n 为2000000，则将使用7813个块。请注意，所有线程块都在向量的不同部分上操作。它们可以以任意顺序执行。程序员不能对执行顺序做出任何假设。具有较少执行资源的小型GPU可能仅以并行方式执行一个或两个这些线程块。较大的GPU可能并行执行64或128个块。这使得CUDA核函数具有硬件执行速度的可伸缩性。也就是说，相同的代码在小型GPU上以较低的速度运行，在大型GPU上以较高的速度运行。我们将在第4章《计算架构和调度》中重新讨论这一点。

需要再次指出，矢量加法示例之所以被用来说明，是因为它简单。实际上，为设备分配内存、从主机传输输入数据到设备、从设备传输输出数据到主机以及释放设备内存的开销可能会使最终代码比图2.4中的原始顺序代码更慢。这是因为相对于处理或传输的数据量，由核函数执行的计算量很小。对于两个浮点输入操作数和一个浮点输出操作数，仅执行了一次加法。真实应用通常具有相对于处理的数据量更多的工作量，这使得额外的开销是值得的。真实应用还倾向于在多个核函数调用之间保持数据在设备内存中，以便摊销开销。我们将介绍几类这类应用的例子。

脚注

- 8. 在这个例子中，我们使用了任意的块大小256，但块大小应由稍后介绍的多个因素确定。

2.7 编译

我们已经看到，实现CUDA C核心需要使用多种不属于C语言的扩展。一旦这些扩展在代码中被使用，传统的C编译器就无法接受它了。代码需要由一个能够识别和理解这些扩展的编译器编译，比如NVCC（NVIDIA C编译器）。如图2.14顶部所示，NVCC编译器处理CUDA C程序，使用CUDA关键字来区分主机代码和设备代码。主机代码是纯粹的ANSI C代码，使用主机的标准C/C++编译器编译，并作为传统的CPU进程运行。带有CUDA关键字的设备代码指定了CUDA核函数及其相关的辅助函数和数据结构，由NVCC编译成称为PTX文件的虚拟二进制文件。这些PTX文件由NVCC的运行组件进一步编译成真实的目标文件，并在支持CUDA的GPU设备上执行。

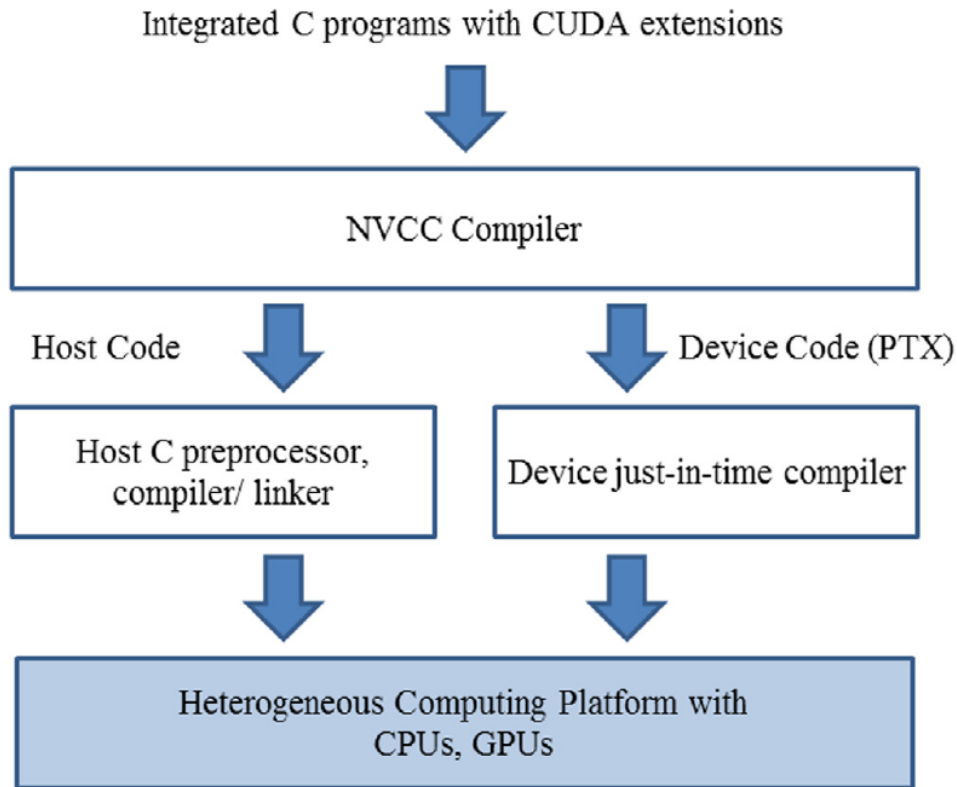


图2.14 CUDA C程序的编译过程概述

2.8 总结

本章提供了CUDA C编程模型的快速、简化概述。CUDA C扩展了C语言以支持并行计算。我们在本章中讨论了这些扩展的基本子集。为方便起见，我们总结了本章中讨论的扩展如下：

2.8.1 函数声明

CUDA C扩展了C函数声明语法，以支持异构并行计算。这些扩展总结在图2.12中。使用“`_global_`”、“`_device_`”或“`_host_`”中的一个，CUDA C程序员可以指示编译器生成内核函数、设备函数或主机函数。所有没有这些关键字的函数声明默认为主机函数。如果在函数声明中同时使用“`_host_`”和“`_device_`”，编译器将为设备和主机分别生成两个版本的函数。如果函数声明没有任何CUDA C扩展关键字，该函数默认为主机函数。

2.8.2 内核调用和网格启动

CUDA C扩展了C函数调用语法，使用由“`<<<`”和“`>>>`”括起的内核执行配置参数。这些执行配置参数仅在调用内核函数以启动网格时使用。我们讨论了定义网格维度和每个块维度的执行配置参数。读者应参阅CUDA编程指南（NVIDIA，2021）以获取有关内核启动扩展以及其他类型执行配置参数的更多详细信息。

2.8.3 内建（预定义）变量

CUDA内核可以访问一组内建的、预定义的只读变量，允许每个线程与其他线程区分开，并确定要处理的数据区域。在本章中，我们讨论了`threadIdx`、`blockDim`和`blockIdx`变量。在第3章“多维网格和数据”中，我们将详细讨论使用这些变量的更多细节。

2.8.4 运行时应用程序编程接口

CUDA支持一组API函数，为CUDA C程序提供服务。我们在本章中讨论的服务是`cudaMalloc`、`cudaFree`和`cudaMemcpy`函数。这些函数由主机代码调用，以代表调用程序分配设备全局内存、释放设备全局内存和在调用程序的代表上在主机和设备之间传输数据。读者请参阅CUDA C编程指南，了解其他CUDA API函数。我们本章的目标是介绍CUDA C的核心概念以及对C的基本扩展，以编写一个简单的CUDA C程序。该章节绝不是所有CUDA功能的全面介绍。这些功能的一些将在本书的其余部分中进行介绍。然而，我们的重点将放在这些功能支持的关键并行计算概念上。我们将只介绍我们代码示例所需的CUDA C功能，用于并行编程技术。总的来说，我们鼓励读者随时查阅CUDA C编程指南，以获取有关CUDA C功能的更多详细信息。

grid

↓

block

↓

wrap (auto-wrapping, usually 32).

↓

thread

global - threadIdx = blockIdx * blockDim + threadIdx.